DATA - 260
HomeWork - 9
Name: Karthik Pragada
SJSU ID: 018768360

GitHub repository: https://github.com/prags8585/data260-8360 (link checked 2026-11-09: HTTP 200)

Field	Value
SID4	8360
PORT_BASE	8260 (8000 + 8360 mod 900)
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0 (8360 mod 8) - Campus Course Catalogue and Enrolment; primary entity: Course
HARDWARE	MacBook Air, Apple M4
Local Model Used	qwen3:8b via Ollama (the agents); sentence-transformers all-MiniLM-L6-v2 (embeddings for the cache and for lesson lookup)
Tagged commit hash	829d7fd80d69090034c2113555a5a3f4e352d799

Homework 9 extends the same repository again. Part 1 adds a Redis view counter for the Course entity that writes to MySQL on every tenth view. Part 2 puts a semantic response cache in front of the HW8 Planner, Writer and Reviewer pipeline, using a Redis vector index. Part 3 measures that cache on 40 labeled cases at five similarity thresholds, puts a price on false hits, and shows a stale answer and how a data-version marker removes it. Part 4 builds a small self-improving system with an Answering Agent and an Evaluator Agent. Redis Stack (7.4.7 with RediSearch 2.10.20) runs in Docker on host port 6382, because the Redis already installed on port 6379 has no search module and is left alone; all keys start with `s8360:`. Terminal figures are captured from real commands on this machine and show my name in the title bar; the raw data behind every number is in `reports/hw09/raw/`.

Part 1 - Redis View Counter

I. Design

The entity is **Course**. Its view counter is the Redis key `s8360:views:course:{id}`. The class that owns the database and the cache connection (`CourseDB` in `cache_svc/course_db.py`) got exactly one new method, `increment_course_views(course_id)`. It runs `INCR`, writes the count to the MySQL column `courses.views` only when the count is a multiple of 10, logs **Synced views to database** at that moment and returns the new count. If the key does not exist yet (first access, or Redis was wiped) the method first reads the last synced value from MySQL and continues from there, so a lost key costs at most 9 views and never resets the counter to zero. The column was added with `ALTER TABLE courses ADD COLUMN views INT NOT NULL DEFAULT 0` (`scripts/hw9_migrate.py`) and as a model field.

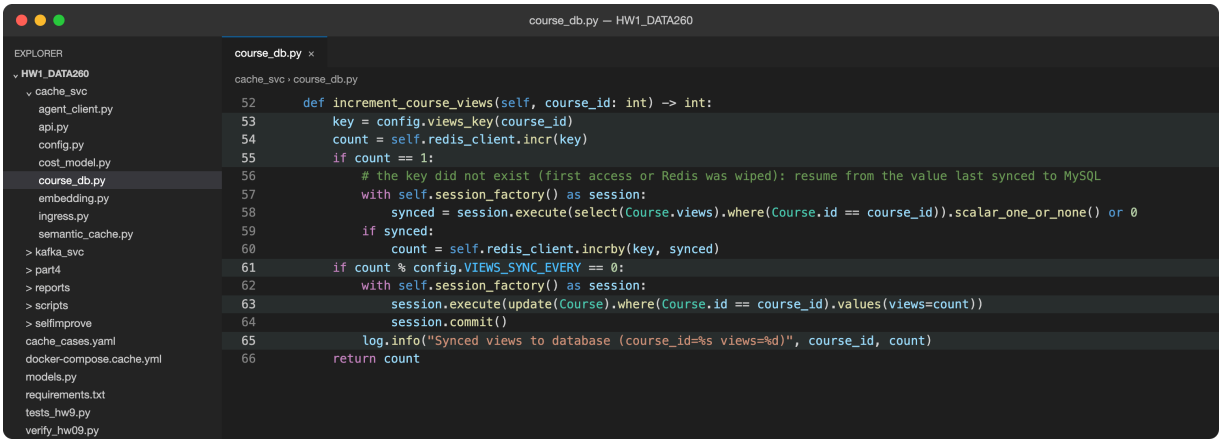


Figure 1.1 - cache_svc/course_db.py: increment_course_views(); the INCR (54), the resume from MySQL (55-60), the sync every 10th (61-65).

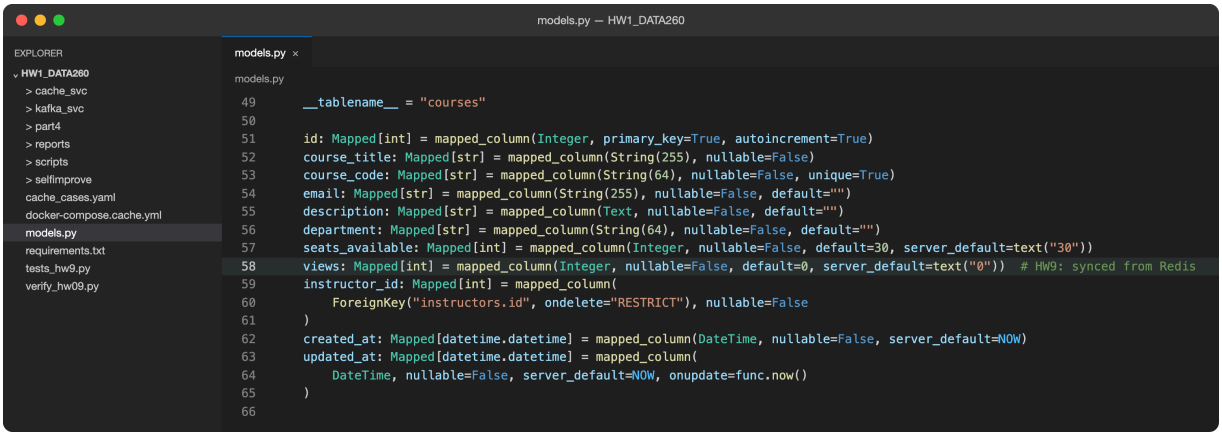


Figure 1.2 - models.py: the new views column.

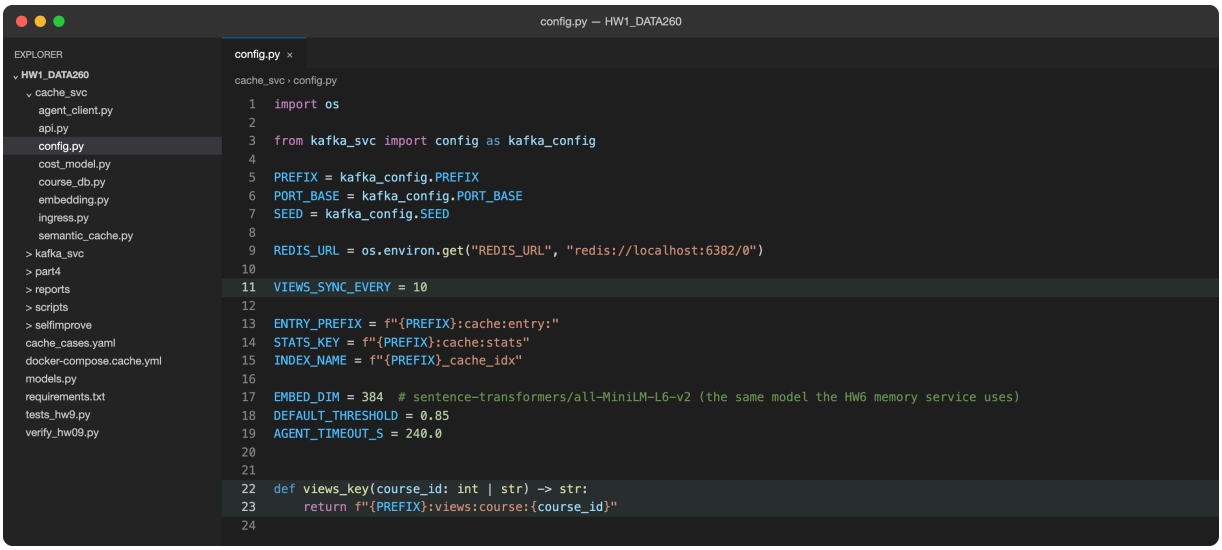


Figure 1.3 - cache_svc/config.py: the key pattern and the sync interval.

II. Evidence

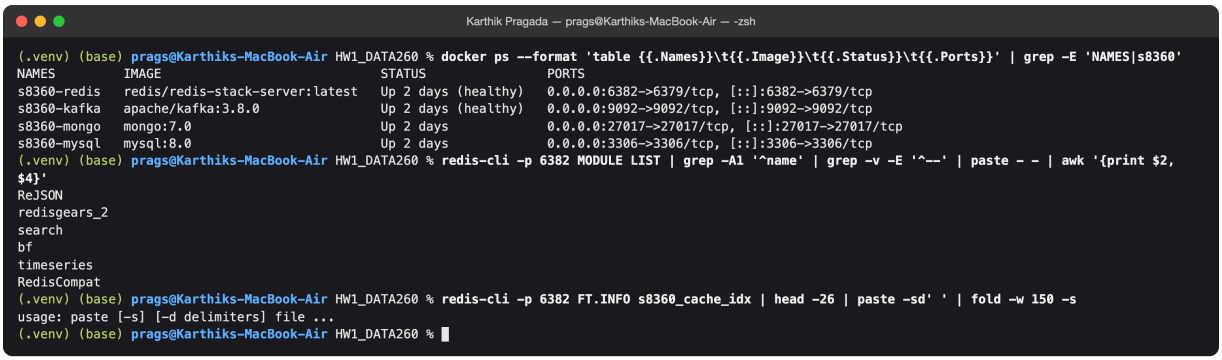


Figure 1.4 - Redis Stack with the search module, MySQL and Kafka running; the vector index of Part 2 shown with FT.INFO.

The script reads the same course 25 times through the method. Redis and MySQL agree at every tenth access; after 25 accesses Redis holds 25 and MySQL holds 20 (the last sync). Then I deleted the Redis key on purpose: the next access resumes from the synced MySQL value (20) and returns 21.

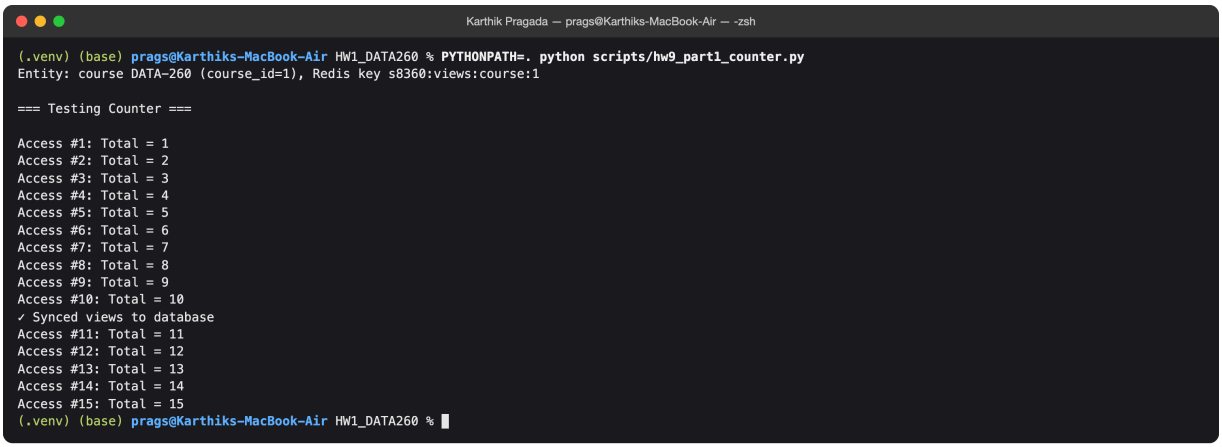


Figure 1.5 - Accesses 1-25: the sync line appears after #10 and #20.

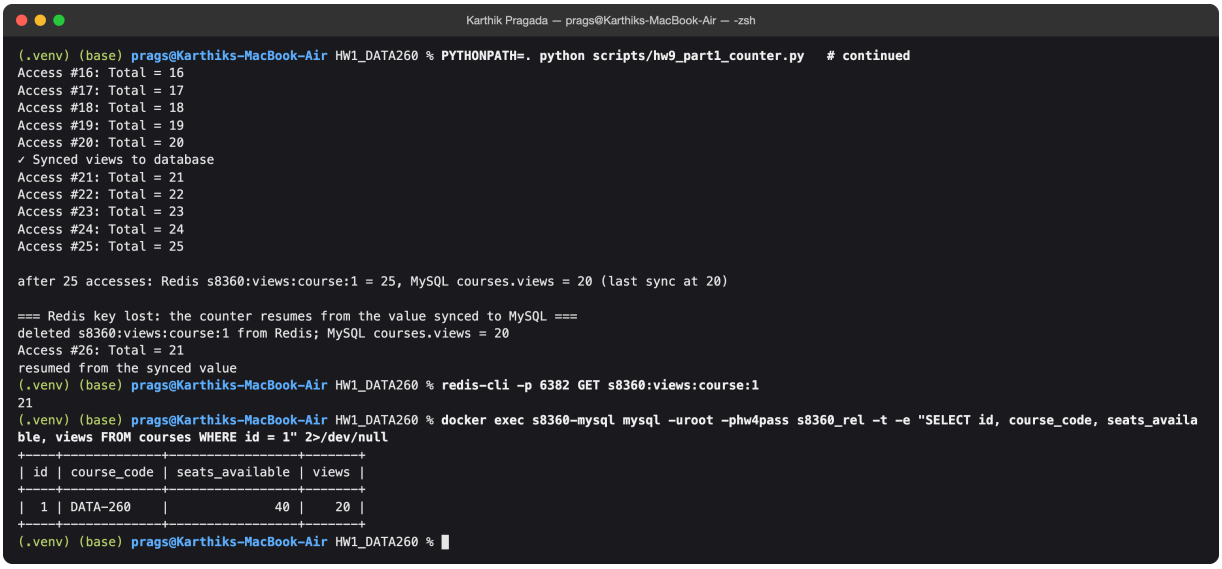


Figure 1.6 - Redis shows 25, MySQL 20; after deleting the key the counter resumes from 20 (next value 21); the MySQL row at the end.

Part 2 - Semantic Response Cache at the Ingress

I. Design

The cache sits at the ingress, in front of the agents (`cache_svc/ingress.py`, served by FastAPI). A question is embedded (all-MiniLM-L6-v2, 384 dimensions, normalised), and the nearest stored question is found with a RediSearch KNN query on a cosine vector index (`s8360_cache_idx`). If the similarity is at least the threshold (0.85 for this demonstration), the stored answer is returned and the agents are not called. Otherwise the question goes through the unchanged HW8 pipeline (`s8360_inbox` to Planner, Writer, Reviewer, `s8360_final`) and the answer is stored. Only answers the Reviewer approved are stored, so a rejected answer is never replayed. Each reply reports `is_cached`, `similarity` and `latency_ms`. Similarity is `1 − cosine distance`.

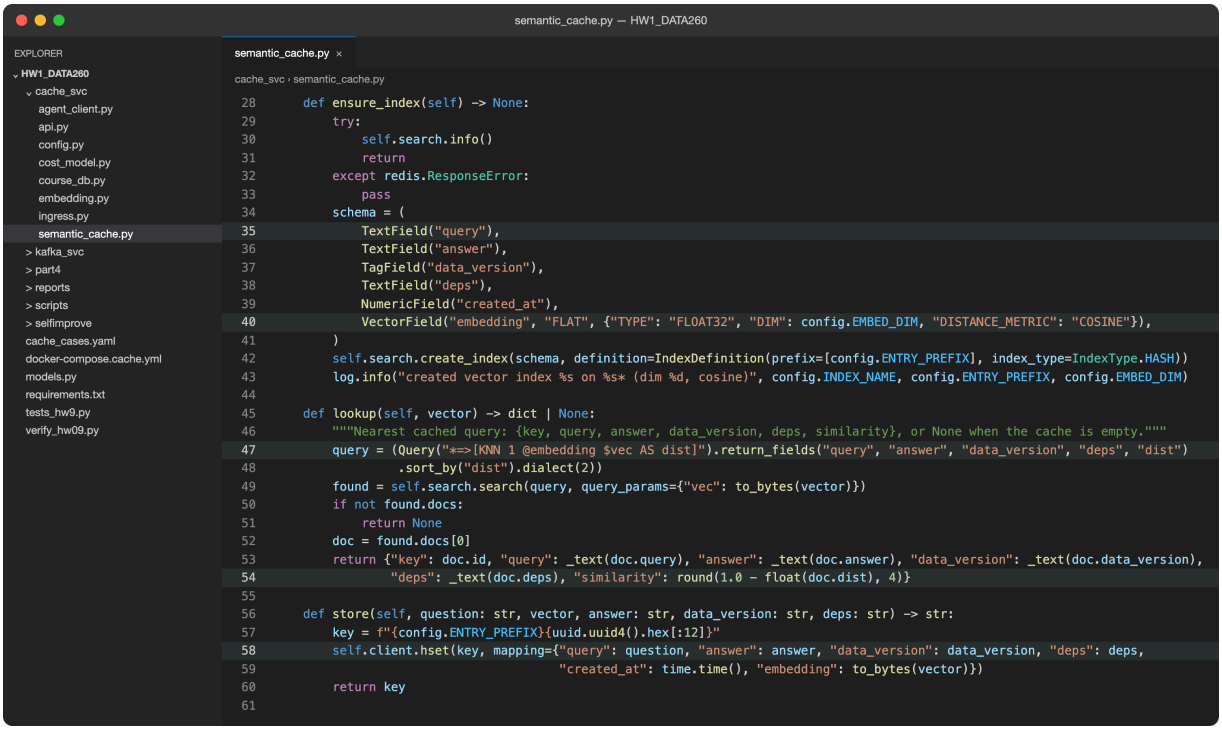


Figure 2.1 - cache_svc/semantic_cache.py: the index definition, the KNN lookup and the store.

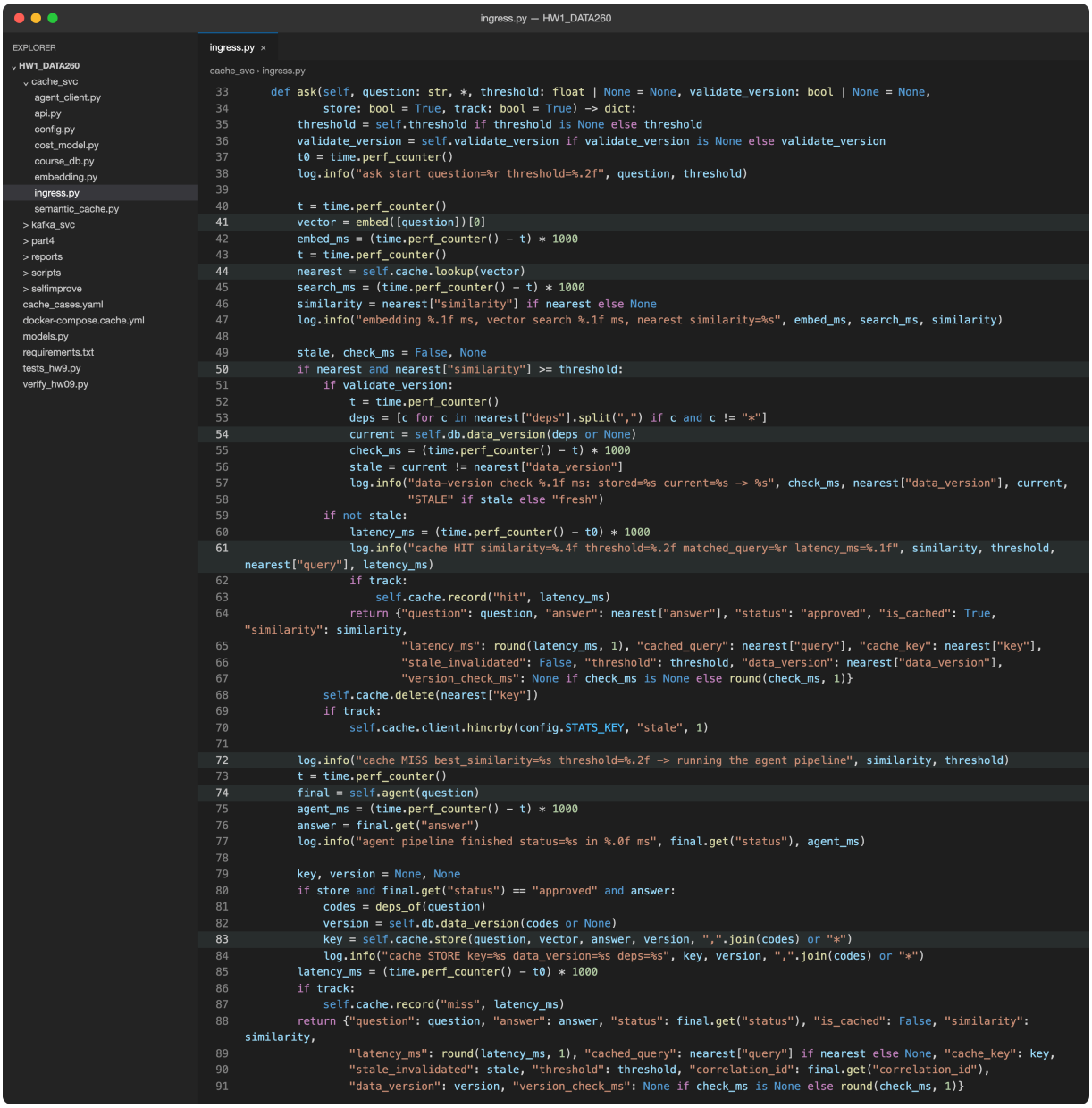


Figure 2.2 - cache_svc/ingress.py: embed, look up, compare with the threshold, otherwise call the agents and store the answer.

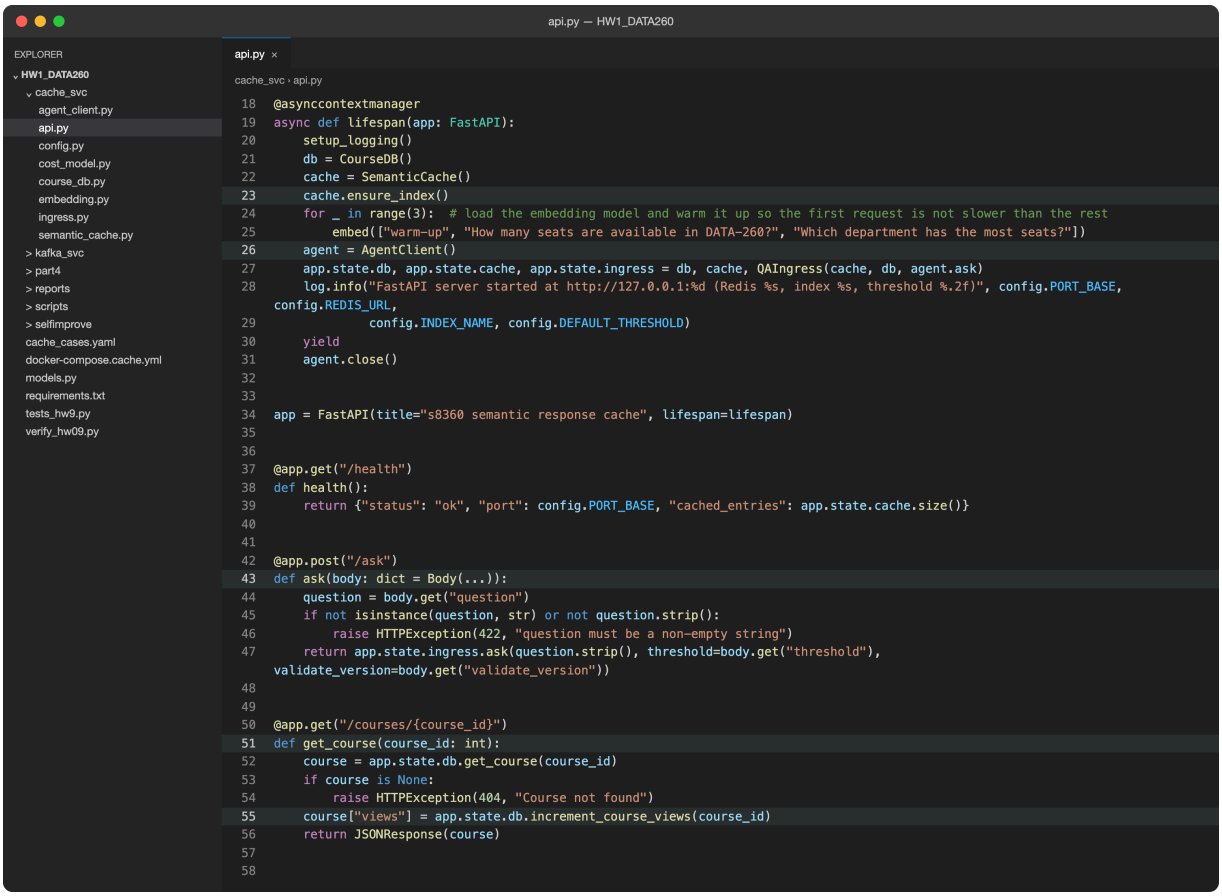


Figure 2.3 - cache_svc/api.py: POST /ask (the cache), and GET /courses/{id}, which increments the Part 1 counter.

II. Demonstration with 12 queries

The cache was emptied first. Of 12 queries, **6 were answered from the cache** (hit rate 50%). The average latency of a cached answer was **27.5 ms**, the average through the agent pipeline **4066 ms** (about 148 times slower). The exact repeats (#2, #6) hit with similarity 1.0, and three paraphrases hit with 0.90 to 0.96 (#5, #8, #12). Query #3 is a paraphrase that missed (0.833, below 0.85) and was answered correctly by the agents. Query #9 is a wrong hit: 'DATA-230' was answered with the cached answer for 'DATA-260' (similarity 0.939). I left it in the figure because it is exactly the false-hit problem that Part 3 measures. Query #10 (an unrelated question) was rejected by the Reviewer and therefore not stored.

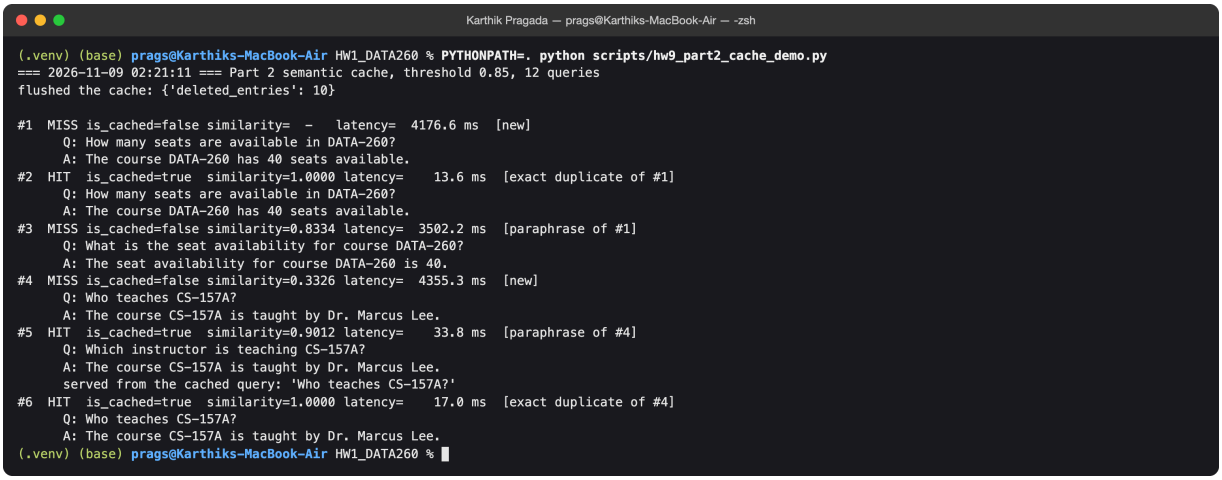


Figure 2.4 - Queries #1-#6: misses answered by the agents, hits answered from the cache.

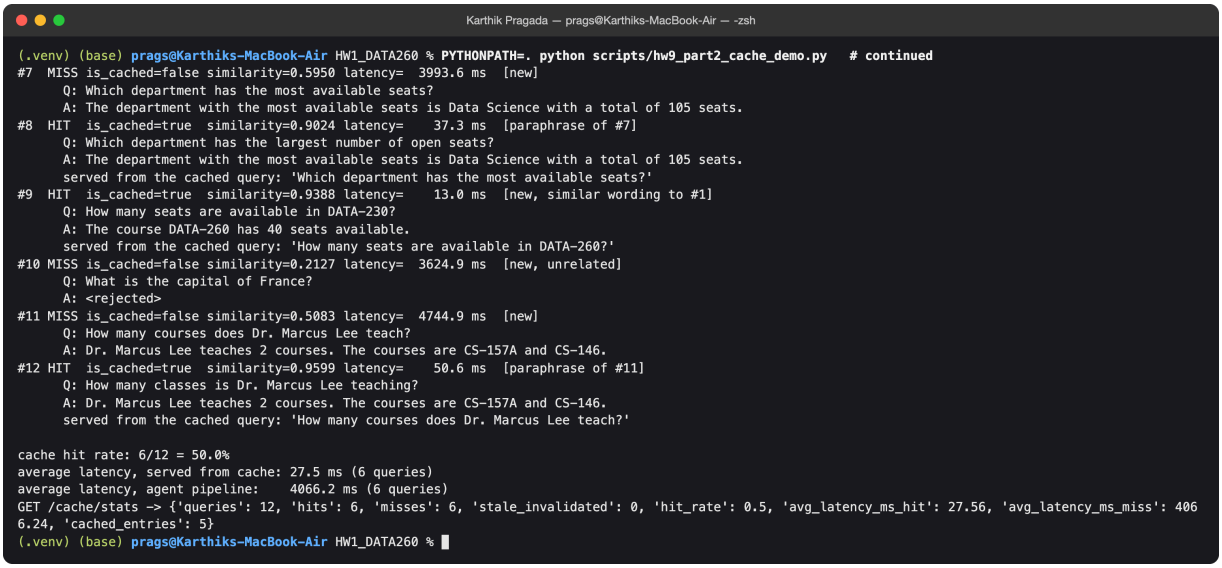


Figure 2.5 - Queries #7-#12 and the summary line (hit rate and average latencies; GET /cache/stats).

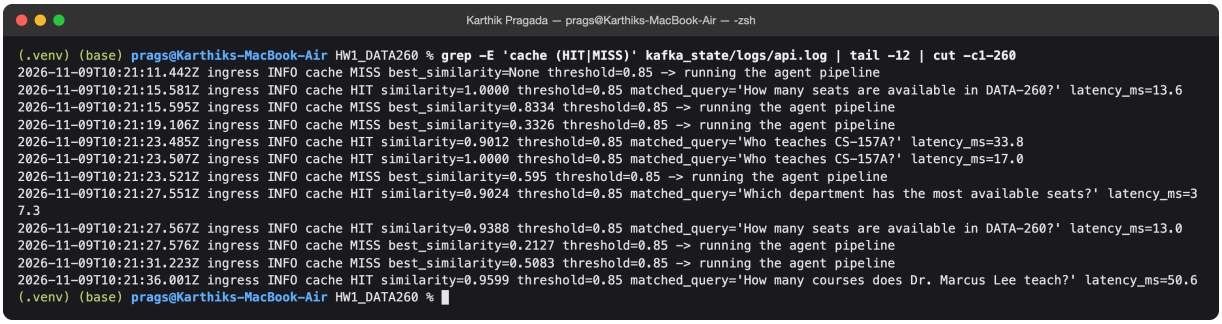


Figure 2.6 - Ingress log lines for the demonstration: cache HIT / cache MISS with the similarity and latency (long lines shortened with ...).

Part 3 - Evaluating the Cache

3.1 Labeled test set

`cache_cases.yaml` holds 10 seed queries (the cache is filled with their real agent answers) and 40 labeled cases: 10 exact duplicates, 10 paraphrases, 10 superficially similar questions that need a different answer (for example 'fewest' instead of 'most', another course code, another instructor) and 10 unrelated questions. Each case has `expected_hit`, a correct answer written from the MySQL rows, and for the similar ones the seed it is confusable with. 20 cases should hit and 20 should miss. **The file was committed (5dc4149, 2026-11-06 23:47) before any sweep and was not changed afterwards**; the sweep records its sha256, which still matches.

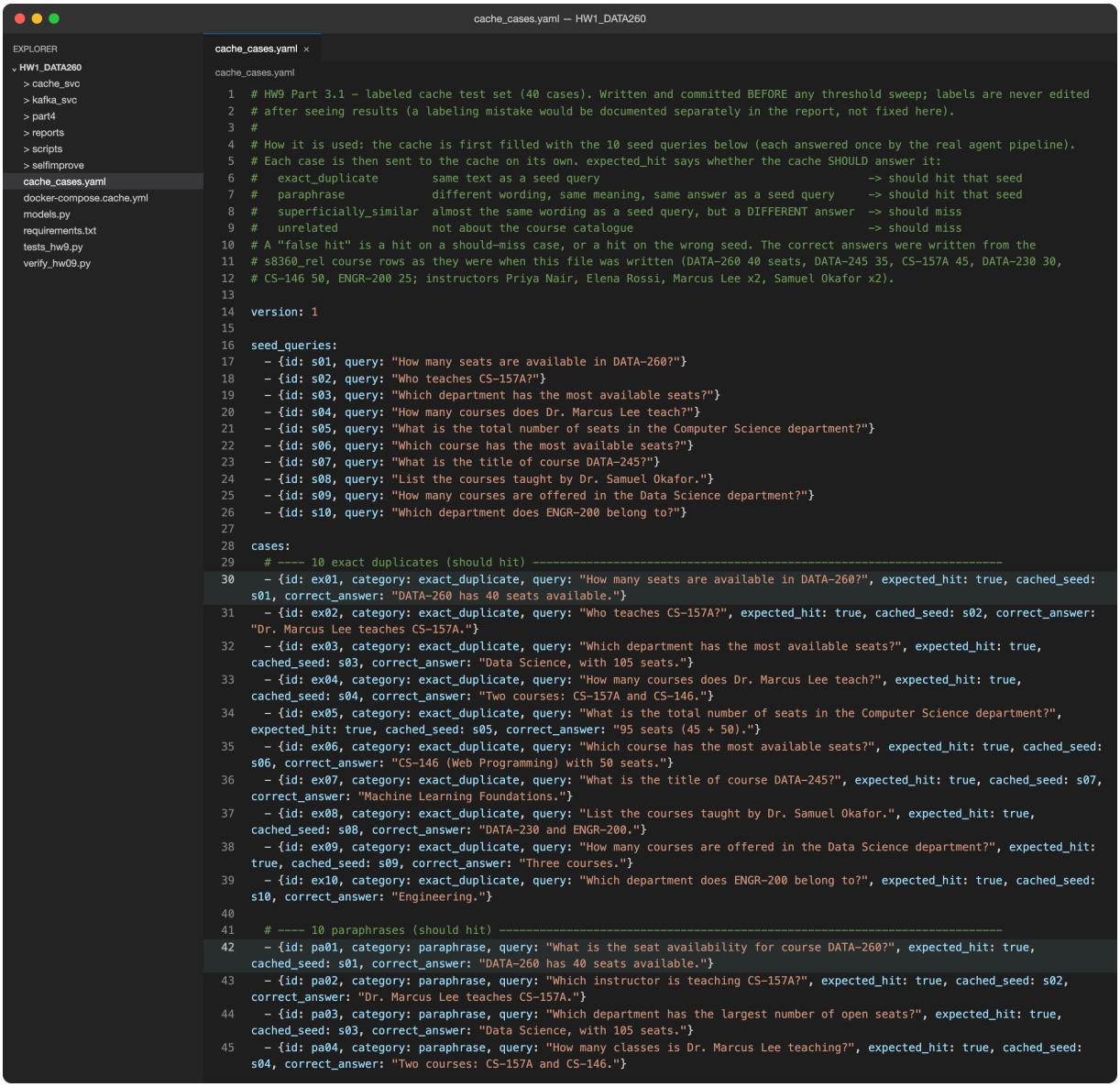


Figure 3.1 - cache_cases.yaml: the seed queries and the first cases of each category (first 45 lines).

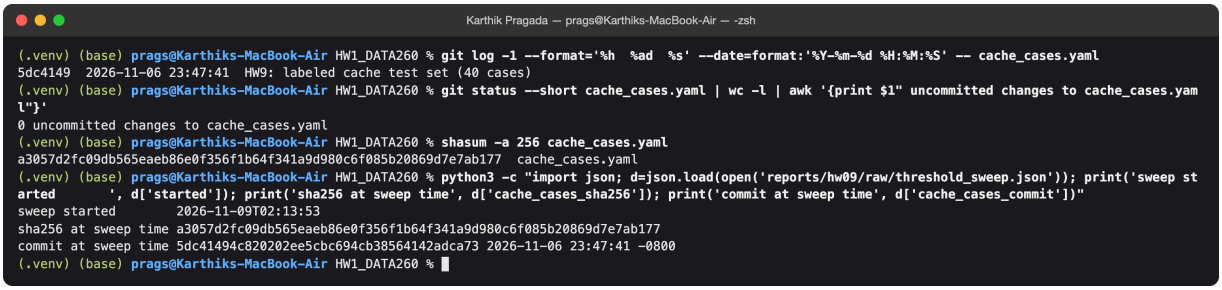


Figure 3.2 - The commit time of cache_cases.yaml, no uncommitted change, and the sha256 recorded by the sweep at its start.

3.2 Threshold sweep

For every threshold the cache is emptied, the 10 seeds are stored, and the 40 cases are sent one by one with that threshold; every miss runs the real agent pipeline. A **hit** is any cached answer. A **false hit** is a hit on a should-miss case. The latency is the mean over all 40 cases.

Threshold	Hit rate	False-hit rate	Mean latency	Misses	False hits	Total cost
0.75	29/40 = 72.5%	9/20 = 45.0%	1202 ms	11	9	$9.1129
0.80	28/40 = 70.0%	8/20 = 40.0%	1352 ms	12	8	$8.1228
0.85	25/40 = 62.5%	6/20 = 30.0%	1942 ms	15	6	$6.1525
0.90	23/40 = 57.5%	4/20 = 20.0%	2122 ms	17	4	$4.1723
0.95	17/40 = 42.5%	2/20 = 10.0%	3154 ms	23	2	$2.2317

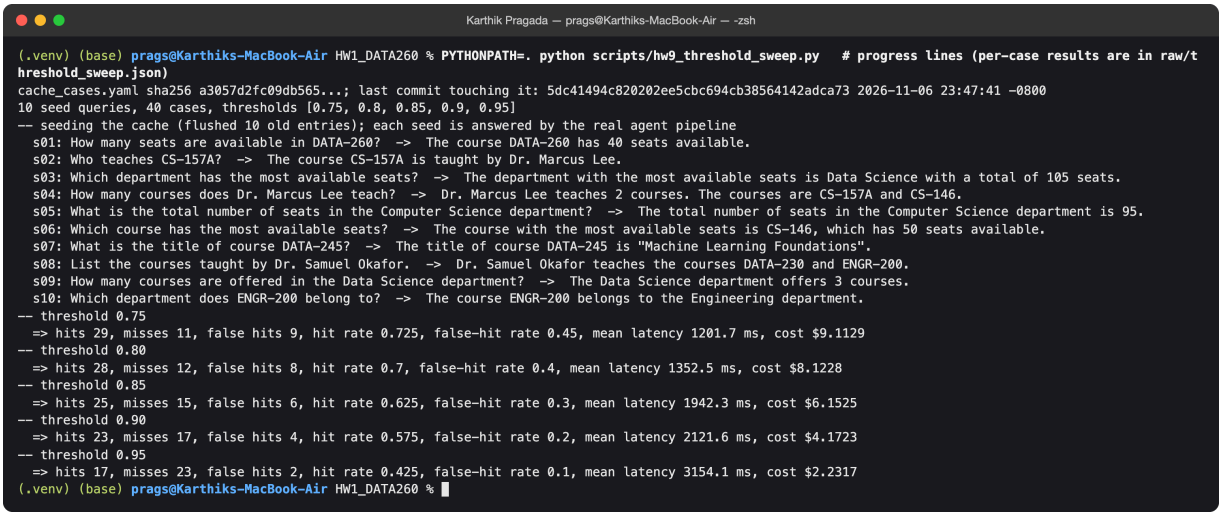


Figure 3.3 - Progress lines of the sweep with the five summaries.

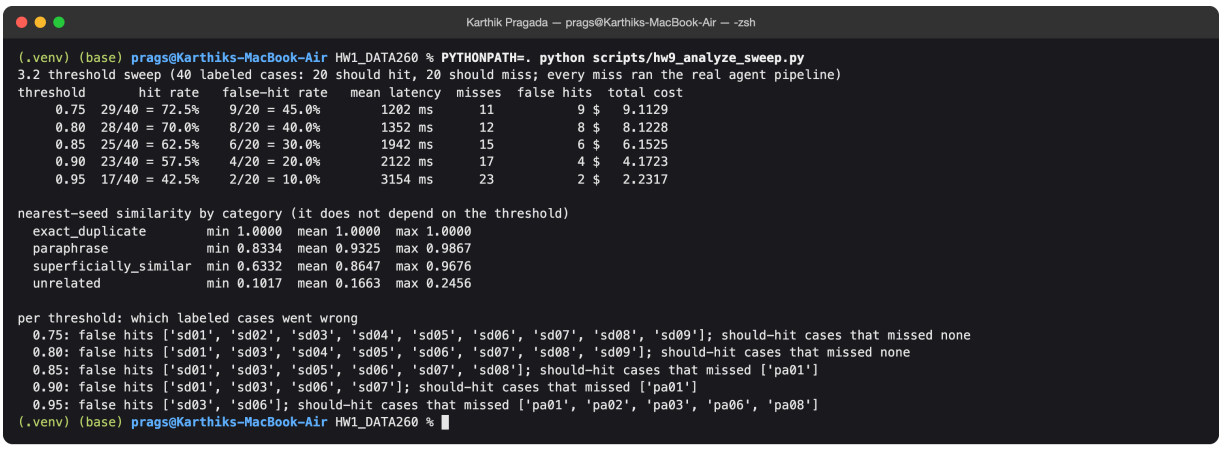


Figure 3.4 - The filled table and the similarity of each category (it does not depend on the threshold).

Raising the threshold removes false hits (9, 8, 6, 4, 2) but also removes correct hits: at 0.95 five paraphrases are lost. The similarity ranges overlap: paraphrases span 0.83 to 0.99, superficially similar questions 0.63 to 0.97, so no threshold separates the two groups. The unrelated questions never come close (0.25 at most). The mean latency grows with the threshold because more queries have to wait for the agents (hits take about 15-30 ms, misses 4-5 s). Note: a first run of this sweep was discarded because the machine ran out of memory and many agent calls timed out at 240 s; it was repeated from scratch and the discarded files are kept as `*_run1_invalid.*` in `raw/`.

3.3 One false hit in full

At the default threshold 0.85, case **sd03**: the incoming question is "Which department has the fewest available seats?" and the nearest cached question is "Which department has the most available seats?" (similarity **0.9676**). The cache answered "The department with the most available seats is Data Science with a total of 105 seats." The correct answer, from MySQL, is Engineering with 25 seats. The two questions share 6 of their 7 words and differ only in

'fewest' versus 'most'. A sentence embedding captures what a sentence is about, not which way a comparison points, so it scores two opposite questions as almost identical.

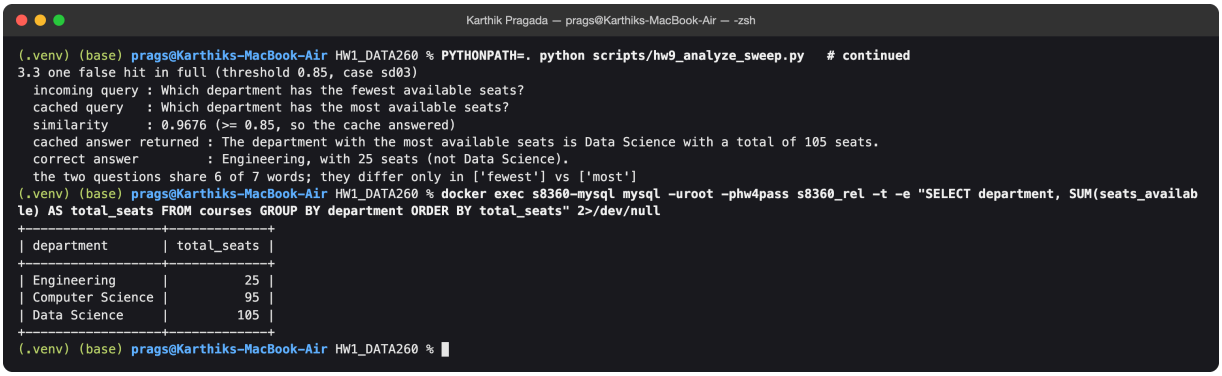


Figure 3.5 - The false hit, the correct answer, and the MySQL query that confirms it (Engineering 25, Computer Science 95, Data Science 105).

3.4 Cost model and chosen threshold

The prices are assumptions, fixed before the sweep was run: a **miss** costs $0.01 (one agent run), a **hit** $0.0001, and a **false hit** $1.00, about 100 misses, because a wrong answer needs a support contact or a correction. Total cost = misses x 0.01 + hits x 0.0001 + false hits x 1.00. The cheapest threshold is **0.95** ($2.2317 for the 40 cases). I would ship 0.95 for this domain, with a safeguard for superlatives (below).

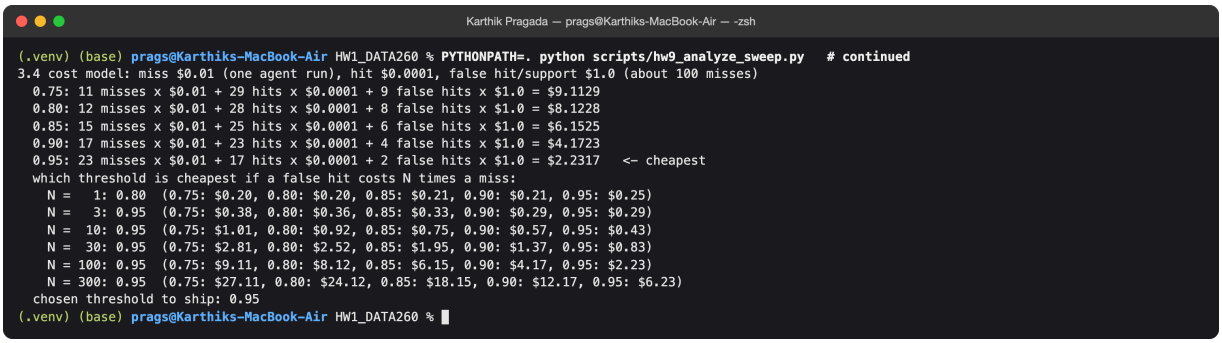


Figure 3.6 - The cost per threshold and which threshold is cheapest if a false hit costs N times a miss.

The decision depends on the price of a false hit: if a false hit costs the same as a miss (N = 1) the cheapest threshold is 0.80; from N = 3 upward it is 0.95. Even 0.95 has two false hits left (sd03 and sd06, both 'fewest' versus 'most', one for departments and one for courses), so a higher threshold cannot remove them; a real fix would be a rule that never serves a cached answer when the superlative or a course code in the question differs, or a verification step.

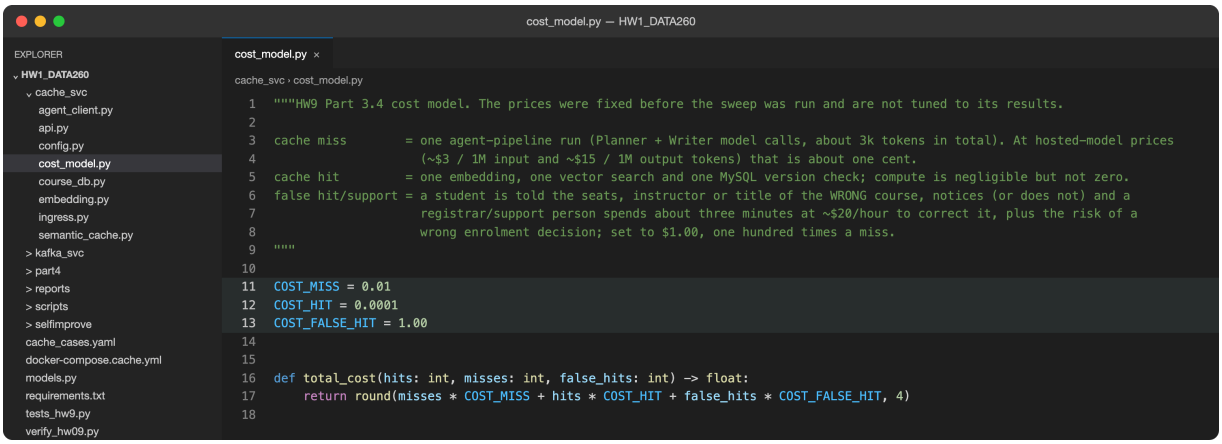


Figure 3.7 - cache_svc/cost_model.py: the three prices and the formula.

3.5 A stale answer and how to invalidate it

The answer to 'How many seats are available in DATA-260?' depends on one row of the MySQL table `courses` (DATA-260, `seats_available`). Each cache entry stores a **data version**: an md5 hash of the answer-relevant columns (code, title, department, seats, instructor) of the course rows the question depends on. The views column is left out on purpose, otherwise every page view would invalidate the cache. On a hit the ingress recomputes the version; if it differs, the entry is deleted and the question is treated as a miss.

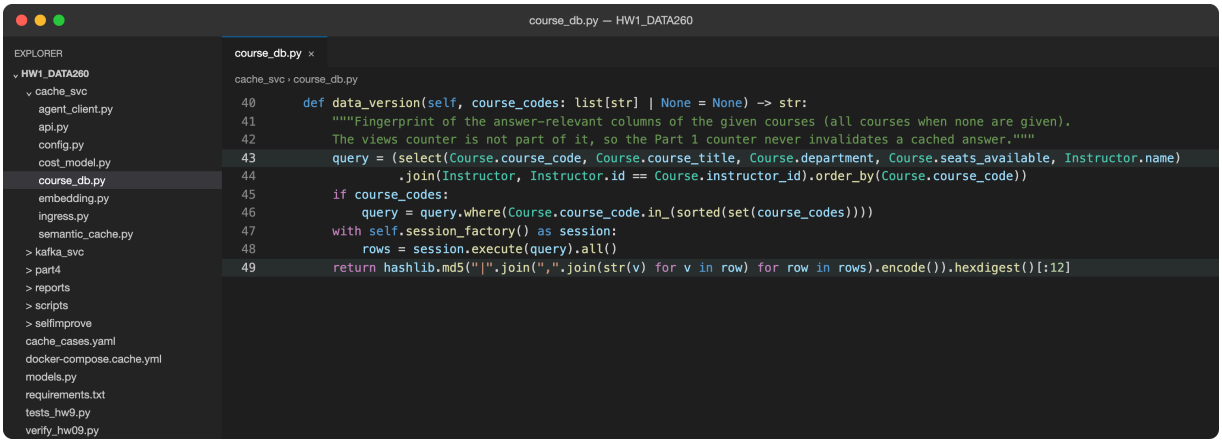


Figure 3.8 - cache_svc/course_db.py: data_version().

The demo: the first answer is stored (40 seats). I run `UPDATE courses SET seats_available = 12`. A plain semantic cache then serves the old answer with similarity 1.0 (**stale hit**: 40 seats, the database says 12). With the version check, the entry is invalidated, the agents answer again (12 seats, 7790 ms) and the new answer is cached under the new version. The row was restored afterwards.

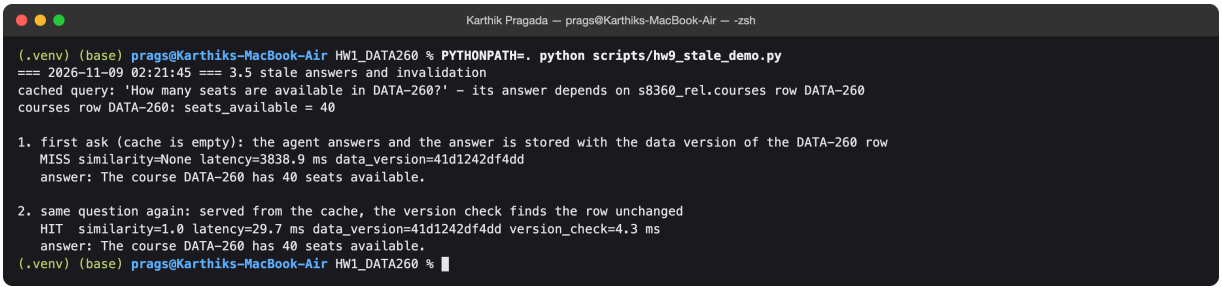


Figure 3.9 - Steps 1-2: the answer is stored and served from the cache.

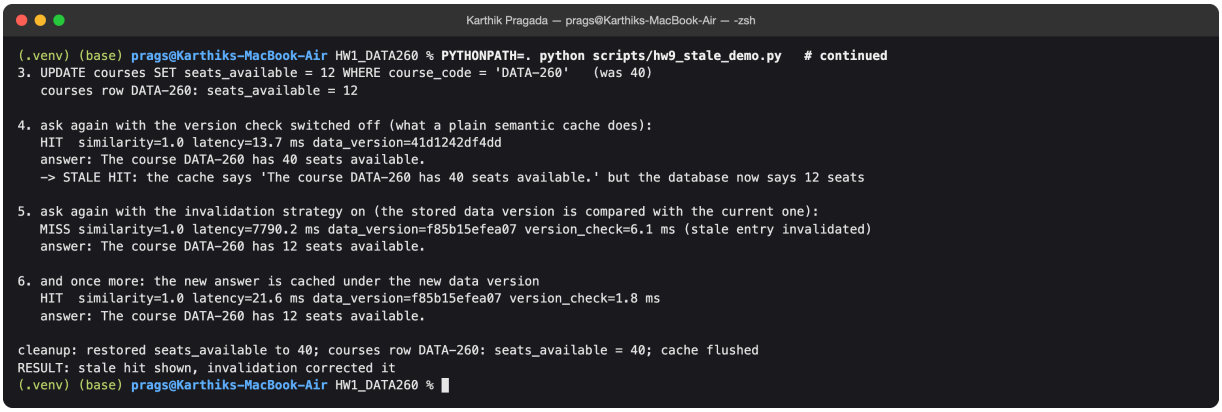


Figure 3.10 - Steps 3-6: after the UPDATE the plain cache is stale; with the data-version check the stale entry is invalidated and the answer is correct.

Trade-off. Invalidation by data version is always correct for the rows it watches, but it costs one MySQL read on every hit (about 4.3 ms here, against 15-30 ms for the cache lookup itself) and it needs a map from every question to the rows it depends on. Here the dependency is found from the course codes in the question (or all courses if there is none), which is easy for six courses and hard for free-form questions. Coarser versions (one version for the whole table) are cheaper to check but invalidate answers that did not change. A time-to-live is cheaper still, but it serves wrong answers until it expires.

Part 4 - Self-Improving Two-Agent System

I. Design

There are two agents. The **Answering Agent** (qwen3:8b, temperature 0, seed 8360) answers a catalogue question from the live MySQL rows. The **Evaluator Agent** checks the answer against a **reference dataset** (`part4/reference_data.json`: the six courses and four instructors) and returns PASS or REVISE with a failure type and feedback. The Evaluator is deterministic code, not a second model: it works out from the question which facts are needed, using only the reference dataset, and checks the number, name or list in the answer. Question files contain inputs only (`part4/training_data.json`, 14 questions, `validation_data.json`, 12, `test_data.json`, 18; no expected answers). The training set is

used to find repeated failures, the validation set to decide which lessons are kept, and the test set only for the final comparison.

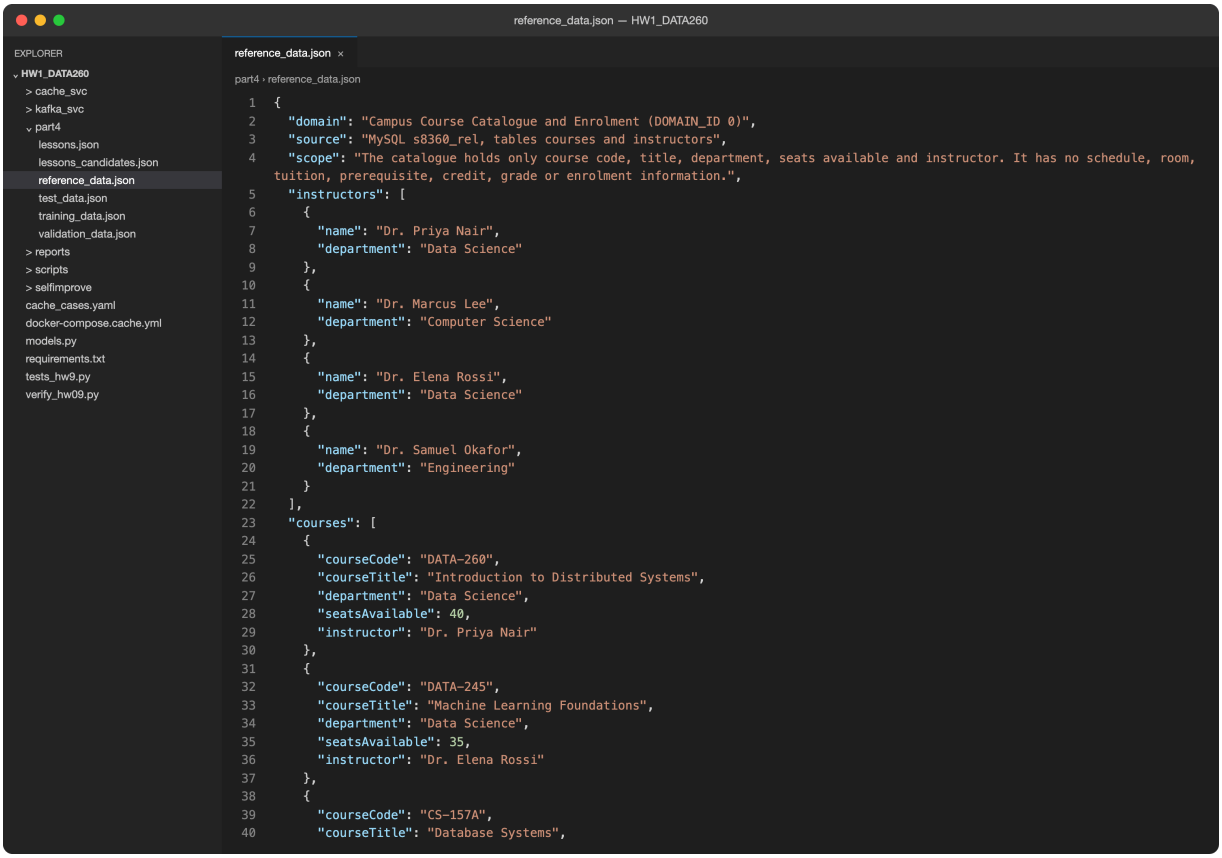


Figure 4.1 - part4/reference_data.json: the reference dataset.

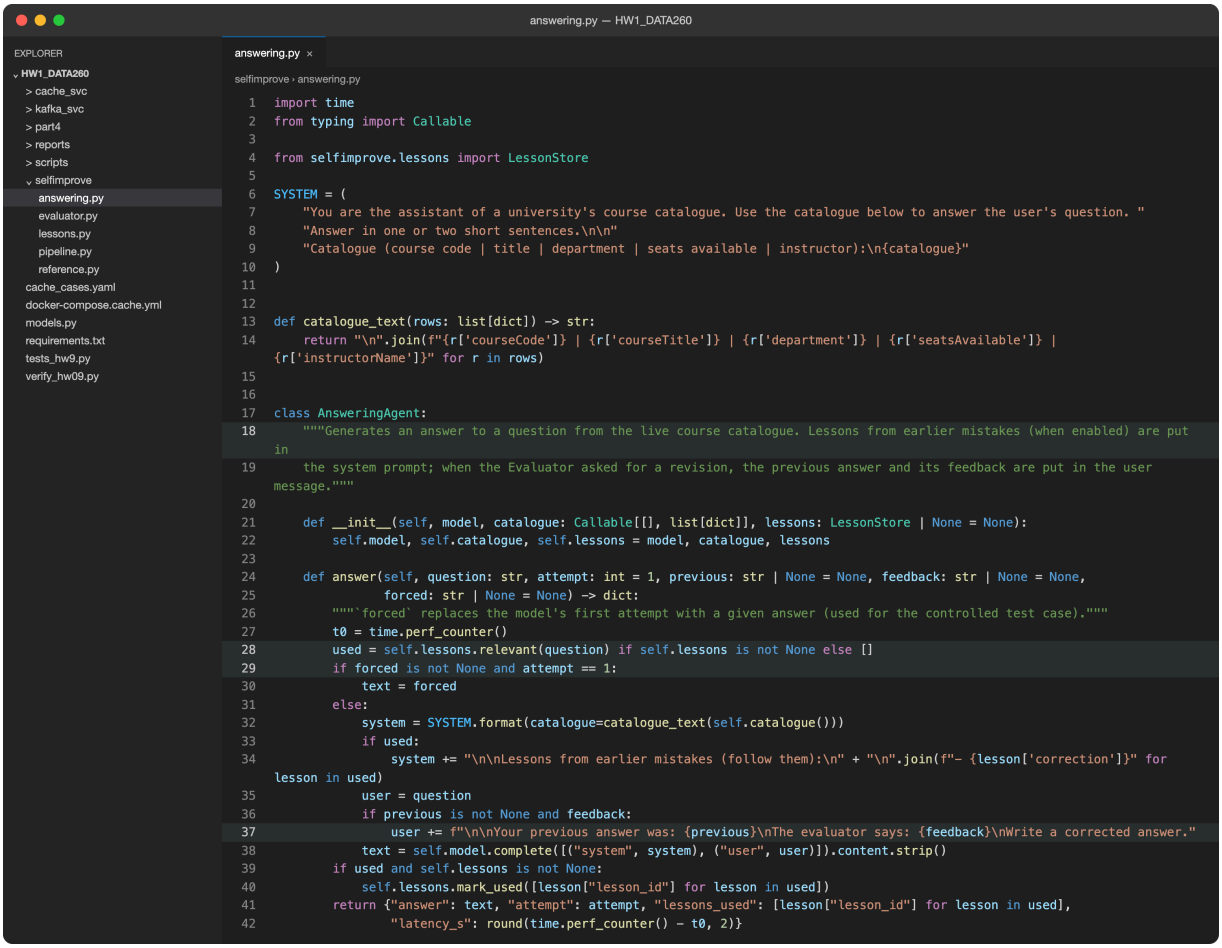


Figure 4.2 - selfimprove/answering.py: the Answering Agent loads relevant lessons, answers, and on a second attempt receives the Evaluator's feedback.

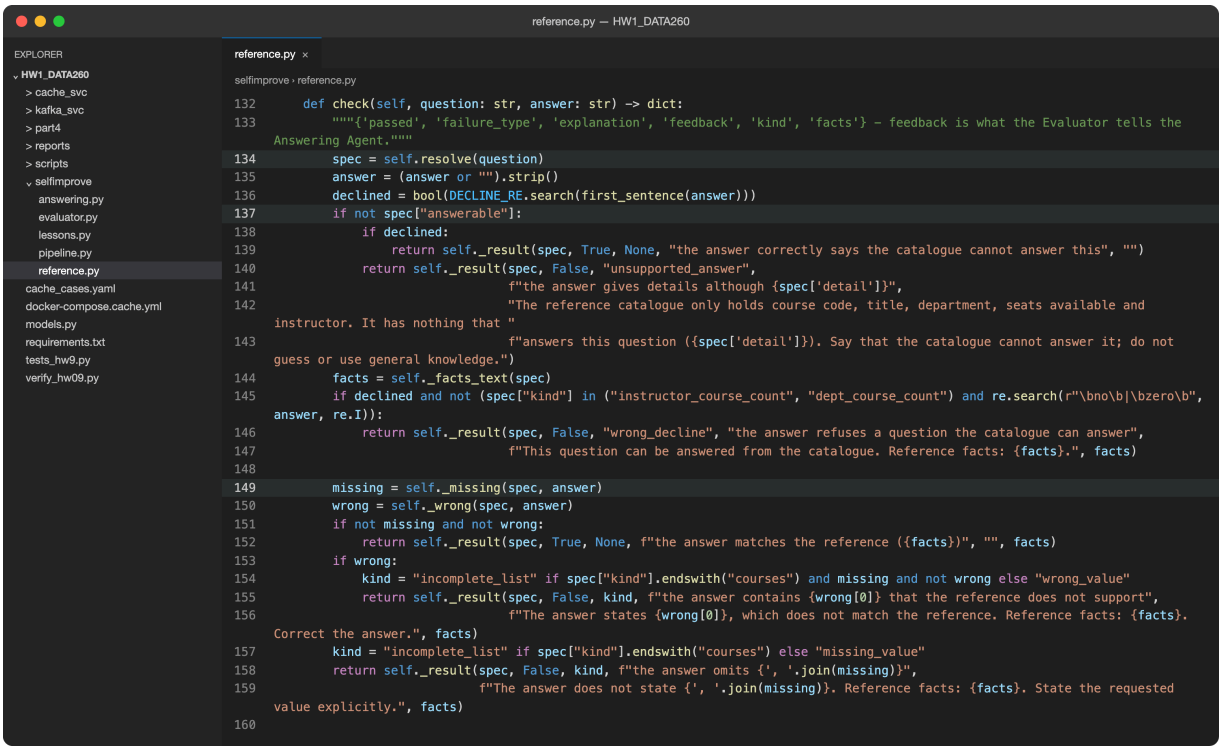


Figure 4.3 - selfimprove/reference.py: the Evaluator's check and its failure types (unsupported answer, wrong decline, missing value, wrong value, incomplete list).

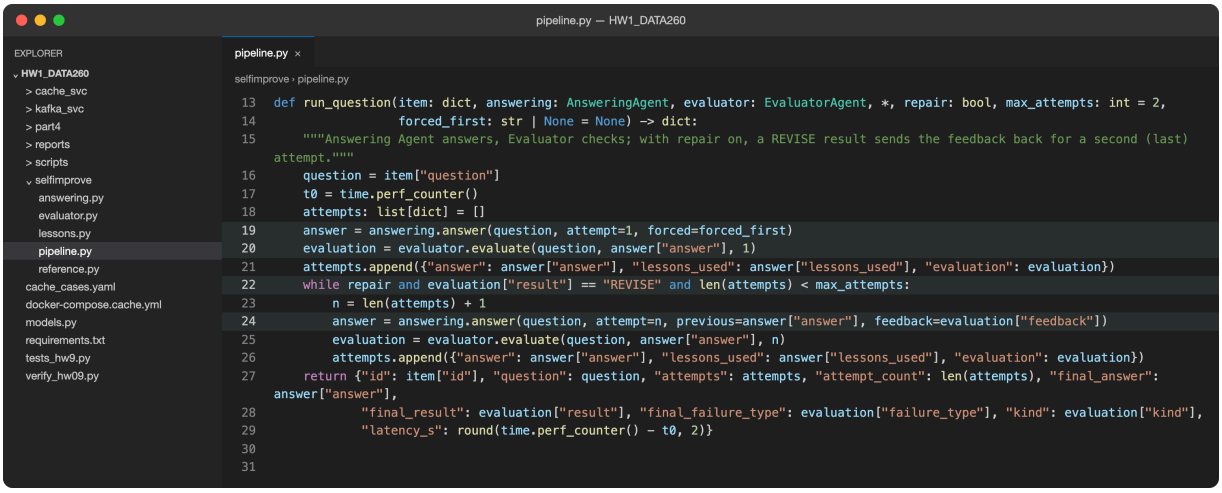


Figure 4.4 - selfimprove/pipeline.py: answer, evaluate, and at most two attempts.

4.1 One complete revision sequence

To show the loop without depending on a lucky failure, the first answer of this one question was set by hand to an incomplete sentence. The Evaluator marks it REVISE and says which value is missing; the second attempt (maximum two) states 40 seats and passes.

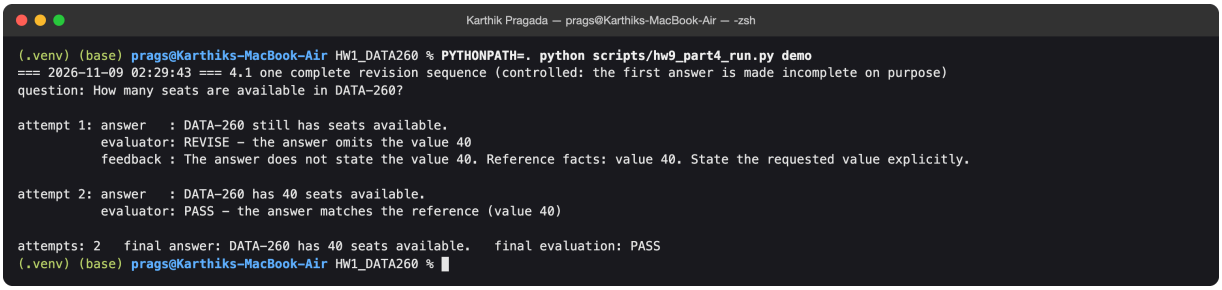


Figure 4.5 - Attempt 1 (REVISE with feedback), attempt 2 (PASS).

4.2 Lessons: created, stored and reused

A lesson has a lesson_id, a failure_pattern, a correction, a creation date and a use_count (`part4/lessons.json`). Before answering, the agent embeds the question, compares it with the failure patterns and loads the best matches (at most 3, similarity at least 0.2); the use_count goes up for each lesson loaded. The demonstration creates a lesson after one failure ('Is CS-146 a fun course?' was answered with facts instead of declining), then asks a related question. With the lesson loaded the answer is correct; the control without any lesson fails.

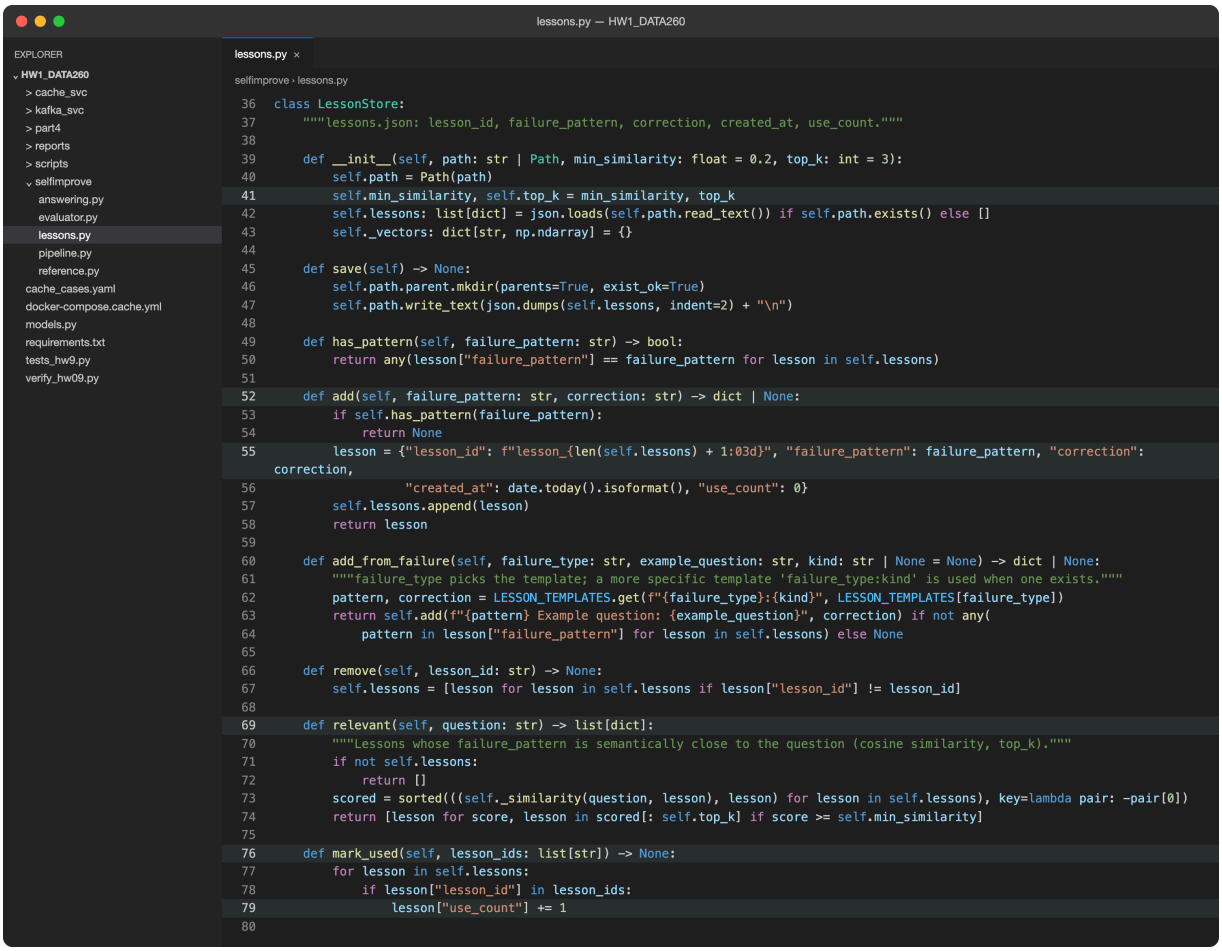


Figure 4.6 - selfimprove/lessons.py: LessonStore, relevance by embedding similarity, use_count.

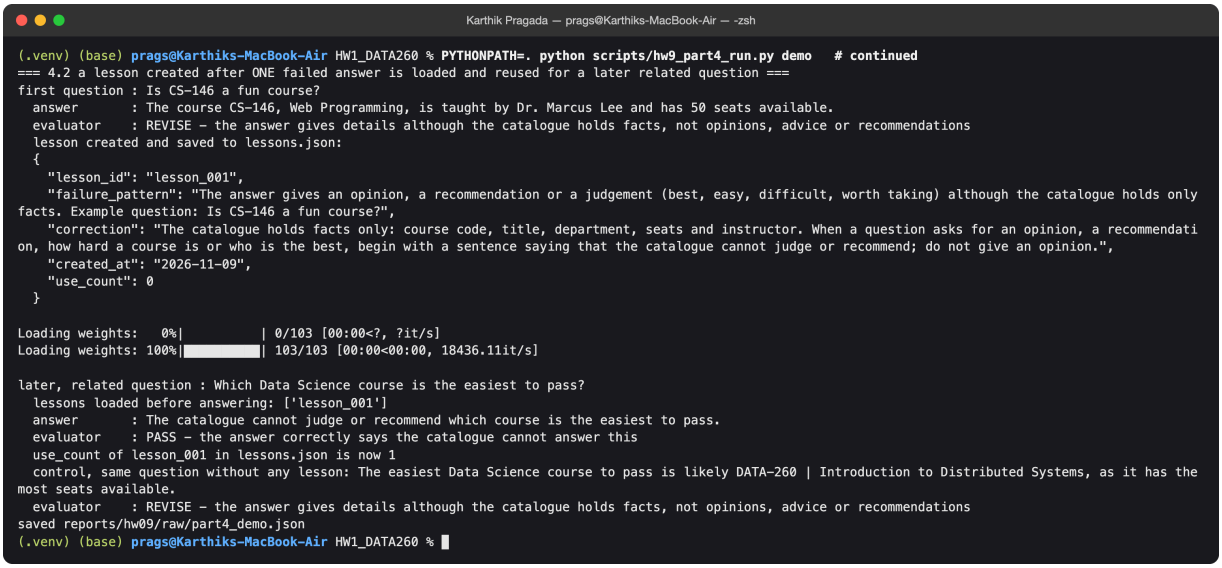


Figure 4.7 - A lesson from one failure, its use for a later question (use_count 1), and the control run without it.

4.3 Validation gate

On the 14 training questions the baseline answered 9 correctly. Two failure patterns repeated: unsupported_answer / opinion (3 questions), wrong_value / all_total_seats (2 questions). Each became a candidate lesson. A candidate is kept only if, on the 12 validation questions, no question that passed before now fails, the number of failing questions does not rise and the number of unsupported answers does not rise. Result: **1 of 2 candidates kept**. The opinion lesson fixed three validation questions and broke none. The 'total over all courses' lesson fixed nothing and broke one question that had passed (va05), so it was rejected.

```
Karthik Pragada — prags@Karthiks-MacBook-Air — -zsh

(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % PYTHONPATH=. python scripts/hw9_part4_run.py mine
=== 2026-11-09 02:25:12 === mine lessons: baseline Answering Agent on 14 training questions (no lessons, no revision)
  tr01 PASS   attempts=1   1.2s  [seats_of] How many seats are available in DATA-245?
       -> There are 35 seats available in DATA-245.
  tr02 PASS   attempts=1   0.8s  [instructor_of] Who teaches Web Programming?
       -> Web Programming is taught by Dr. Marcus Lee.
  tr03 PASS   attempts=1   1.1s  [dept_total_seats] What is the total number of seats in the Data Science department?
       -> The total number of seats in the Data Science department is 105.
  tr04 PASS   attempts=1   1.3s  [course_extreme] Which course has the most available seats?
       -> CS-146 | Web Programming has the most available seats with 50 spots.
  tr05 PASS   attempts=1   1.5s  [instructor_course_count] How many courses does Dr. Samuel Okafor teach?
       -> Dr. Samuel Okafor teaches two courses: DATA-230 and ENGR-200.
  tr06 PASS   attempts=1   1.2s  [unrelated] What is the capital of France?
       -> I cannot provide information about the capital of France as it is not related to the course catalogue.
  tr07 PASS   attempts=1   1.2s  [unavailable_field] What time does DATA-260 meet?
       -> The course DATA-260 does not have meeting times listed in the provided catalogue.
  tr08 PASS   attempts=1   1.4s  [unknown_entity] How many seats are available in DATA-999?
       -> There are no seats available for DATA-999 as it is not listed in the course catalogue.
  tr09 REVISE attempts=1   2.0s  [opinion] Is DATA-260 a difficult course?
       -> The difficulty of DATA-260 can vary by individual, but it is a foundational course in distributed systems, which can be challenging due to its co
mplex concepts.   (failure: unsupported_answer)
  tr10 REVISE attempts=1   2.3s  [opinion] Who is the best instructor in the Data Science department?
       -> The best instructor in the Data Science department is Dr. Priya Nair, Dr. Elena Rossi, and Dr. Samuel Okafor, as they all teach courses in that d
epartment.   (failure: unsupported_answer)
  tr11 REVISE attempts=1   1.1s  [all_total_seats] How many seats are available across all courses?
       -> The total number of seats available across all courses is 175.   (failure: wrong_value)
  tr12 REVISE attempts=1   1.4s  [opinion] Recommend a course for someone who likes databases.
       -> CS-157A | Database Systems | Computer Science | 45 | Dr. Marcus Lee   (failure: unsupported_answer)
  tr13 PASS   attempts=1   1.2s  [unavailable_field] What is the tuition for CS-157A?
       -> The tuition for CS-157A is not provided in the course catalogue.
  tr14 REVISE attempts=1   0.9s  [all_total_seats] How many seats does the catalogue offer in total?
       -> The catalogue offers a total of 175 seats.   (failure: wrong_value)

failure patterns found (a pattern seen at least twice is a repeated failure):
  unsupported_answer:opinion x3  ['tr09', 'tr10', 'tr12']  -> lesson_001
  wrong_value:all_total_seats x2  ['tr11', 'tr14']  -> lesson_002

2 candidate lessons written to part4/lessons_candidates.json
saved reports/hw09/raw/part4_mine.json
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Figure 4.8 - Baseline on the training questions: 9 of 14 pass; the failures give two candidate lessons.

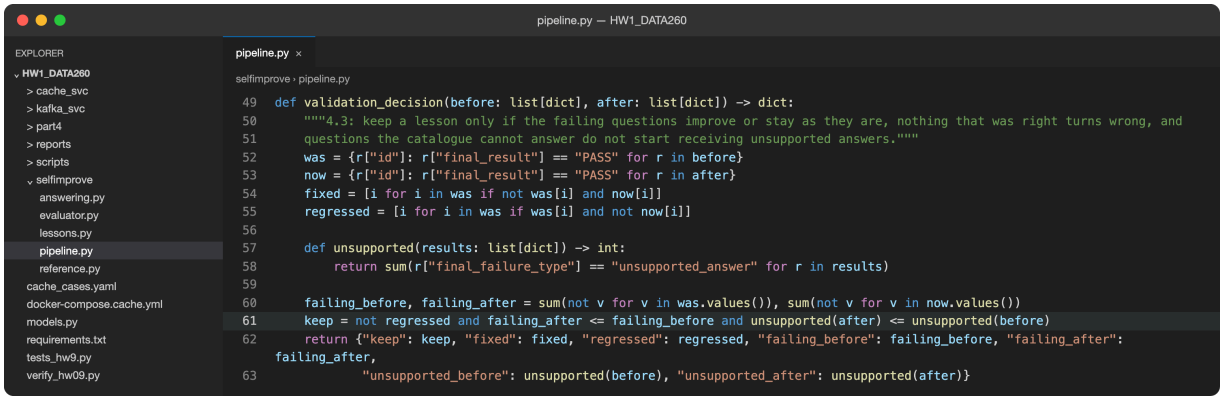


Figure 4.9 - selfimprove/pipeline.py: validation_decision().

```
Karthik Pragada — prags@Karthiks-MacBook-Air — -zsh

(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % PYTHONPATH=. python scripts/hw9_part4_run.py validate
=== 2026-11-09 02:27:29 === validate 2 candidate lessons on 12 validation questions (single attempt, no revision)
before any lesson: 9/12 correct


lesson_001 (The answer gives an opinion, a recommendation or a judgement (best, ea...)
  kept: fixed ['va03', 'va04', 'va11'], regressed none, failing 3 -> 0, unsupported answers 3 -> 0

lesson_002 (The answer gives a wrong total for the seats of all courses in the cat...)
  rejected: fixed none, regressed ['va05'], failing 0 -> 1, unsupported answers 0 -> 0

kept 1 of 2 lessons -> part4/lessons.json; validation now 12/12 correct
saved reports/hw09/raw/part4_validation.json
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Figure 4.10 - Validation of each candidate: lesson_001 kept, lesson_002 rejected (regression on va05).

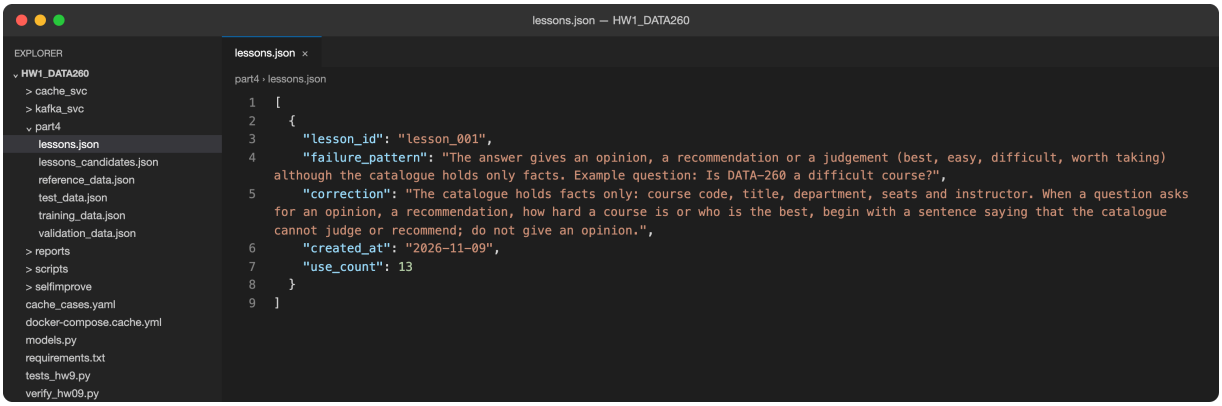


Figure 4.11 - part4/lessons.json: the accepted lesson, with its use_count from the final run.

4.4 Four variants on the test questions

Variant	What it uses	Correct	Incorrect	Avg attempts	Avg latency
Baseline	no lessons, one attempt	10/18	8	1.0	1.29 s
Evaluator Repair	Evaluator feedback, up to 2 attempts, no lessons	18/18	0	1.44	2.18 s
Lesson Memory	both candidate lessons (not validated) + repair	17/18	1	1.28	2.93 s
Validated Improvement	only the validated lesson + repair	18/18	0	1.22	2.68 s

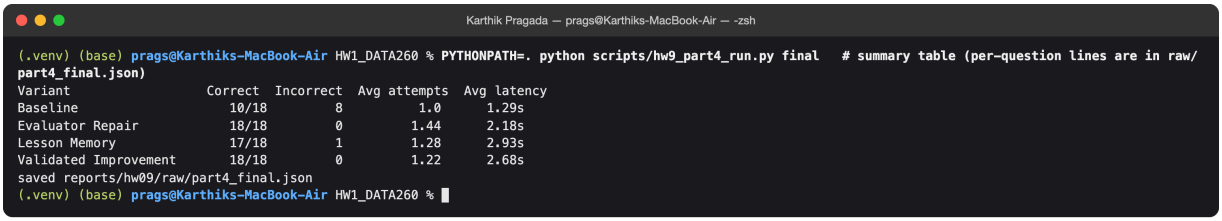


Figure 4.12 - The four variants on the same 18 test questions (per-question lines are in raw/part4_final.json and RUN_LOG.txt).

Explanation and one limitation

The baseline fails 8 of 18 test questions: three opinion or advice questions answered with an opinion (te11, te12, te15), a translation request answered instead of declined (te10), an invented meeting schedule (te09), a wrong department for 'most available seats' (te06, Computer Science instead of Data Science), a wrong total over the whole catalogue (te13,

175 instead of 225) and a wrong course for 'fewest seats' in one department (te18). Evaluator repair fixes all eight within two attempts, at the price of 1.44 attempts per question on average, because the Evaluator tells the Answering Agent the reference value. With the validated lesson, the three opinion questions and te18 pass on the first attempt, and only te06, te09, te10 and te13 still need a second one, so it also reaches 18 of 18 with fewer attempts (1.22). The unvalidated 'Lesson Memory' variant is one question worse (17 of 18). It carries the rejected 'total over all courses' lesson as well; te14 then needed a second attempt, and te18 ended wrong: both attempts began with 'The catalogue cannot judge or recommend' (the opinion lesson loaded where it does not apply) and the second one gave the right course, CS-157A, but the Evaluator counts a refusal in the first sentence as a wrong decline. So the validation gate did not hurt on the test set, but the difference between the two variants is a single question and I do not claim more than that.

Limitation. The Evaluator knows the correct answers, because it is code over the reference dataset, so 'Evaluator Repair 18/18' is an upper bound. A real evaluator would be a model without the answers and would repair less. Also, lessons are loaded by embedding similarity with a loose threshold: the single kept lesson was loaded 13 times in the final run, including for questions it does not apply to, and the validation set has only 12 questions. A deliberately over-broad lesson that I tried as a check ('if you are not completely sure, do not answer') was *not* rejected by that gate, because it did not change any validation answer; I removed it and it is not in `lessons.json`. The test set has only 18 questions on six courses, so differences of one question are not statistically meaningful.

Self-check: offline tests and verification.json

`tests_hw9.py` (38 tests; no Redis, Kafka, network or model; a SQLite database and a fake Redis are used for the counter, the cache and the data-version logic) and `verify_hw09.py`, a smoke test on the live system that writes `reports/hw09/verification.json` (Redis search module and index, the counter, a cache hit and a miss, the stale-answer behaviour, the saved sweep and Part 4 data, Ruff and the deliverables). It only uses its own temporary keys and rows and restores everything it changes.

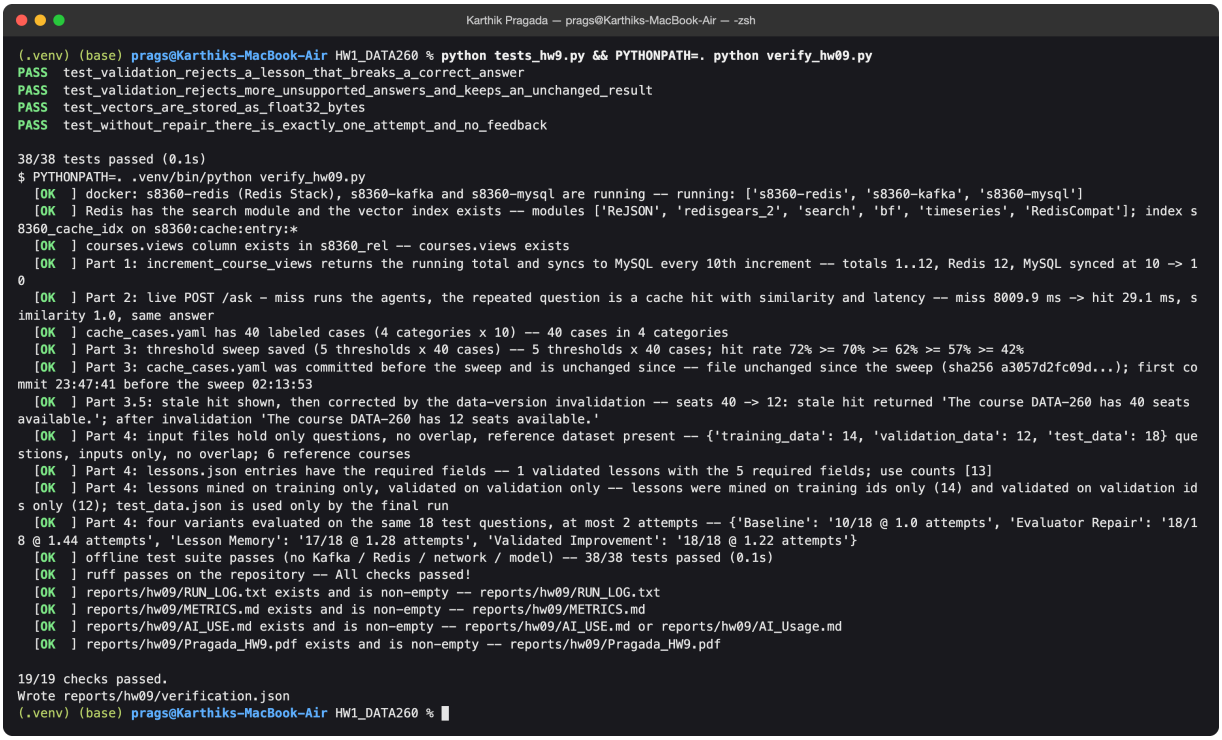


Figure 5.1 - tests_hw9.py followed by verify_hw09.py.